

Machine Learning Lab Report

Lab Report

CSE-816

Submitted By:

Sadia Afroj
ID: 21701031
8th Semester
Department of CSE
University of Chittagong

Submitted To:

Prof. Dr. Mohammad Osiur Rahman
Professor
Department of Computer Science
and Engineering
University of Chittagong

Submitted September 14, 2026

Contents

1 Problem 1: K-Nearest Neighbors (KNN) Based Student Performance Classification 4

1.1 Objective . . . 4
1.2 Input Features and Output . . . 4
1.3 Algorithm Used . . . 4
1.4 Dataset Description . . . 4
1.5 Flowchart . . . 5
1.6 Mathematical Formulation . . . 6
1.6.1 Normalization . . . 6
1.6.2 Distance Calculation . . . 6
1.7 Theory and Background . . . 6
1.8 Implementation (C Code) . . . 7
1.9 Graphical User Interface . . . 9
1.10 Test Cases and Results . . . 9
1.11 Results and Analysis . . . 10
1.12 Conclusion . . . 10

2 Problem 2: Support Vector Machine (SVM) Based Loan Approval Prediction 11

2.1 Objective . . . 11
2.2 Input and Output . . . 11
2.3 Algorithm Used . . . 11
2.4 Dataset Description . . . 12
2.5 SVM Workflow . . . 12
2.6 Mathematical Formulation . . . 13
2.6.1 Decision Function . . . 13
2.6.2 Hinge Loss . . . 13
2.6.3 Normalization . . . 13
2.7 Training Process . . . 13
2.8 Theory and Background . . . 13
2.9 Implementation (C Code) . . . 14
2.10 Graphical User Interface . . . 16
2.11 Test Cases . . . 16
2.12 Results and Analysis . . . 16
2.13 Conclusion . . . 17

3 Problem 3: K-Means Student Performance Clustering **18**
3.1 Objective . . . 18
3.2 Input and Output . . . 18
3.3 Algorithm Used . . . 18
3.4 Dataset Description . . . 18
3.5 Workflow . . . 19
3.6 Mathematical Formulation . . . 20
3.6.1 Distance Calculation . . . 20
3.6.2 Centroid Update . . . 20
3.6.3 Cluster Labeling . . . 20
3.7 Theory and Background . . . 20
3.8 Implementation (C Code) . . . 21
3.9 Graphical User Interface . . . 23
3.10 Test Cases . . . 23
3.11 Results and Analysis . . . 24
3.12 Conclusion . . . 24

4 Problem 4: Decision Tree Based Weather Activity Prediction **25**
4.1 Objective . . . 25
4.2 Input and Output . . . 25
4.3 Algorithm Used . . . 25
4.4 Dataset Description . . . 25
4.5 Workflow . . . 26
4.6 Mathematical Formulation . . . 27
4.6.1 Gini Impurity . . . 27
4.6.2 Weighted Gini for a Split . . . 27
4.7 Theory and Background . . . 27
4.8 Implementation (C Code) . . . 27
4.9 Graphical User Interface . . . 30
4.10 Test Cases . . . 30
4.11 Results and Analysis . . . 31
4.12 Conclusion . . . 31

5 Problem 5: Perceptron Based Apple vs Orange Recognition **32**
5.1 Objective . . . 32
5.2 Input and Output . . . 32
5.3 Algorithm Used . . . 32
5.4 Workflow . . . 33
5.5 Mathematical Formulation . . . 33

5.5.1 Net Input 33
5.5.2 Activation Function 33
5.5.3 Weight Update Rule 33
5.6 Theory and Background 34
5.7 Training Process 34
5.8 Implementation (C Code) 34
5.9 Graphical User Interface 36
5.10 Test Cases 37
5.11 Results and Analysis 38
5.12 Conclusion 38

1 Problem 1: K-Nearest Neighbors (KNN) Based Student Performance Classification

1.1 Objective

The objective of this problem is to implement the K-Nearest Neighbors (KNN) algorithm to classify a student as **PASS** or **FAIL** based on study hours, attendance percentage, and previous examination marks.

1.2 Input Features and Output

The system uses three numerical input features, as shown in Table 1.

Table 1: Input Features

Feature	**Description**
Study Hours	Average daily/regular study hours
Attendance	Attendance percentage
Previous Mark	Previous examination mark

The output is one of two classes:

- PASS
- FAIL

1.3 Algorithm Used

K-Nearest Neighbors (KNN) is a supervised machine learning classification algorithm. For a new data point, it identifies the K closest points in the training data and assigns the class based on the majority vote among those neighbors.
Example: For $K = 5$, if the nearest five students have results {PASS, PASS, FAIL, PASS, PASS}, then PASS occurs 4 times and FAIL occurs once. Since PASS is the majority, the new student is classified as PASS.

1.4 Dataset Description

The dataset file `student_dataset.csv` contains 100 student records with three input features and one target class (Result). A sample of the dataset is shown in Table 2.

Table 2: Sample of the Student Dataset (first 10 records)

Study Hours	Attendance (%)	Previous Mark	Result
1.0	45	30	Fail
1.2	48	32	Fail
1.5	50	35	Fail
1.8	52	38	Fail
2.0	55	40	Fail
6.0	80	66	Pass
6.2	81	67	Pass
7.0	84	70	Pass
9.0	92	78	Pass
11.0	98	86	Pass

Records: **Features:** 3 **Classes:** PASS, FAIL

1.5 Flowchart

The overall working procedure of the system can be summarized as follows:

START

↓

Load Student Dataset

↓

Normalize Features

↓

Enter New Student Data

↓

Calculate Distances to All Training Samples

↓

Sort by Distance

↓

Select K Nearest Neighbors

↓

Majority Voting

↓

Display PASS or FAIL

↓

END

1.6 Mathematical Formulation

1.6.1 Normalization

Since Study Hours, Attendance, and Previous Mark have different numerical ranges, Min-Max normalization is applied to scale all features to the range $[0, 1]$:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \tag{1}$$

1.6.2 Distance Calculation

The similarity between the new student and each training sample is measured using the (squared) Euclidean distance:

$$d^2 = (x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 \tag{2}$$

where x_1, x_2, x_3 are the normalized features of the new student and y_1, y_2, y_3 are the normalized features of a training student. The squared distance is used instead of the square root since it preserves the same neighbor ordering while simplifying computation.

1.7 Theory and Background

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm mainly used for classification and regression tasks. In classification, KNN determines the class of a new data point by examining the K closest training samples. The similarity between samples is commonly measured using Euclidean distance. Before calculating the distance, numerical features can be normalized so that features with larger numerical ranges do not dominate the distance calculation.

In this experiment, KNN is used to classify students into two categories: PASS and FAIL. The classification is based on study hours, attendance percentage, and previous examination marks. For a new student, the algorithm calculates the distance from all training students, selects the nearest K students, and assigns the class based on majority voting.

1.8 Implementation (C Code)

The algorithm was implemented in C using the Win32 API to build a graphical user interface. Key functions are shown below.

Listing 1: Min-Max Normalization Function

```

#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STUDENTS 200

#define ID_STUDY        101
#define ID_ATTENDANCE   102
#define ID_MARK         103
#define ID_K            104
#define ID_PREDICT      105
#define ID_CLEAR        106
#define ID_RESULT       107
#define ID_STATUS       108

typedef struct
{
    double studyHours;
    double attendance;
    double previousMark;
    int result;
} Student;
//////////main//////
double normalize(double value, double min, double max)
{
    if (max == min)
        return 0.0;

    return (value - min) / (max - min);
}
```

Listing 2: Squared Euclidean Distance Calculation

```
return
    (x1 - y1) * (x1 - y1) +
```

```
    (x2 - y2) * (x2 - y2) +
    (x3 - y3) * (x3 - y3);
```

Listing 3: Majority Voting Among K Neighbors

```
for (i = 0; i < k; i++)
{
    if (neighbors[i].result == 1)
        passCount++;
    else
        failCount++;
}

if (passCount > failCount)
    return 1;
else
    return 0;
```

The complete source code (`main.c`) and dataset (`student_dataset.csv`) are included in the project folder submitted alongside this report.

1.9 Graphical User Interface

A Windows GUI was developed so that the user can provide input without using the command line. The interface contains input fields for Study Hours, Attendance, Previous Mark, and K value, along with PREDICT and CLEAR buttons, and a result display area.

1.10 Test Cases and Results

Table 3: Test Cases

Study Hours	Attendance (%)	Previous Mark	K	Prediction
2.0	55	40	5	FAIL
8.0	88	75	5	PASS
10.0	95	82	5	PASS
5.5	78	64	5	Borderline (depends on nearest neighbors)

1.11 Results and Analysis

The implemented KNN system successfully classified students based on their study hours, attendance, and previous examination marks. The results show that students with lower study hours, lower attendance, and lower previous marks tend to be classified as FAIL, while students with higher values are generally classified as PASS. The KNN algorithm makes the prediction based on the similarity between the new student and the existing training samples.

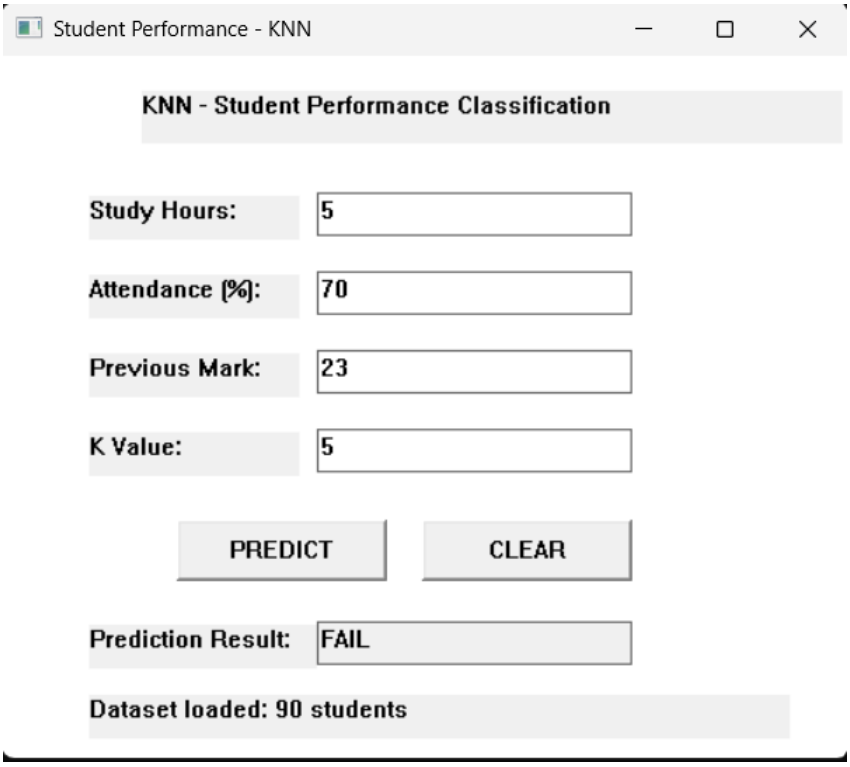


Figure 1: Decision boundary separating Approved and Rejected applicants

1.12 Conclusion

In this experiment, the K-Nearest Neighbors (KNN) algorithm was successfully implemented for student performance classification. The system uses study hours, attendance percentage, and previous examination marks to classify a student as PASS or FAIL. A Windows-based GUI was developed so that users can provide input without using command-line interaction. The experiment demonstrates how KNN uses distance measurement and majority voting to perform classification.

2 Problem 2: Support Vector Machine (SVM) Based Loan Approval Prediction

2.1 Objective

The objective of this problem is to implement a Support Vector Machine (SVM) classifier to predict whether a loan application should be **APPROVED** or **REJECTED** based on the applicant's income and credit score.

2.2 Input and Output

- **Input:** Income (thousand), Credit Score
- **Output:** LOAN APPROVED or LOAN REJECTED

2.3 Algorithm Used

Support Vector Machine (SVM) is a supervised machine learning classification algorithm. SVM attempts to find a decision boundary (hyperplane) between two classes such that the margin between the classes is maximized. In this problem, the two classes are represented as:

- Approved $\rightarrow +1$
- Rejected $\rightarrow -1$

2.4 Dataset Description

The dataset file `loan_dataset.csv` contains 100 applicant records with two input features (Income, Credit Score) and one target class (Result). A sample of the dataset is shown in Table 4.

Table 4: Sample of the Loan Dataset (first 10 records)

Income (thousand)	**Credit Score**	**Result**
20	500	Rejected
22	520	Rejected
24	530	Rejected
27	550	Rejected
30	570	Rejected
50	670	Approved
55	685	Approved
65	720	Approved
80	780	Approved
95	840	Approved

Records: **Features:** Income, Credit Score **Classes:** Approved, Rejected

2.5 SVM Workflow

START

$\downarrow$

Load Loan Dataset

$\downarrow$

Normalize Features

$\downarrow$

Train SVM (find decision boundary)

$\downarrow$

Enter Applicant Information

$\downarrow$

Calculate Decision Value

$\downarrow$

Decision Value ≥ 0?

YES $\rightarrow$ APPROVED NO $\rightarrow$ REJECTED

2.6 Mathematical Formulation

2.6.1 Decision Function

$$f(x) = w_1 x_1 + w_2 x_2 + b \tag{3}$$

where x_1 is the normalized Income, x_2 is the normalized Credit Score, w_1, w_2 are weights, and b is the bias. If $f(x) \geq 0$, the application is Approved; otherwise it is Rejected.

2.6.2 Hinge Loss

$$Loss = \max(0,\ 1 - y \cdot f(x)) \tag{4}$$

where $y = +1$ for Approved and $y = -1$ for Rejected.

2.6.3 Normalization

Since Income (≈20–97) and Credit Score (≈500–850) have very different scales, Min-Max normalization is applied:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \tag{5}$$

2.7 Training Process

The model was trained using batch gradient updates based on the hinge-loss condition, with the following hyperparameters:

- Learning Rate = 0.01
- C (regularization parameter) = 1.0
- Epochs = 5000

2.8 Theory and Background

Support Vector Machine (SVM) is a supervised machine learning algorithm commonly used for classification problems. SVM attempts to find an optimal decision boundary that separates different classes while maximizing the margin between them. The data points closest to the decision boundary are called support vectors, and they play an important role in determining the position of the separating boundary.

In this experiment, SVM is used to predict loan approval based on two features: applicant income and credit score. Approved and rejected applications are represented by positive and negative class labels respectively. The numerical features are normalized before training to ensure that the difference in feature scales does not affect the classification process.

2.9 Implementation (C Code)

The algorithm was implemented in C using the Win32 API to build a graphical user interface. Key functions are shown below.

Listing 4: SVM Decision Function

```

#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_APPLICANTS 200

#define ID_INCOME       201
#define ID_CREDIT       202
#define ID_PREDICT      203
#define ID_CLEAR        204
#define ID_RESULT       205
#define ID_STATUS       206

/* ---------- DATA STRUCTURE ---------- */

typedef struct
{
    double income;
    double creditScore;
    int approved;
} Applicant;


/* ---------- GLOBAL VARIABLES ---------- */

Applicant dataset[MAX_APPLICANTS];

int applicantCount = 0;

/* SVM parameters */
double weightIncome = 0.0;
double weightCredit = 0.0;
double bias = 0.0;

```

```
37
38 /* ============================================================
39    LOAD DATASET
40    ============================================================ */
41
42 int loadDataset()
43 {
44     FILE *file;
45     char line[200];
46     char result[30];
47
48     file = fopen("loan_dataset.csv", "r");
49
50     if(file == NULL)
51     {
52         return 0;
53     }
54 ////main////
55 decision =
56     w1 * x1 +
57     w2 * x2 +
58     bias;
59
60 if (decision >= 0)
61     return 1;
62
63 return 0;
```

Listing 5: Hinge-Loss Based Weight Update

```
1 decision = w1 * x1 + w2 * x2 + bias;
2 margin = y * decision;
3
4 if (margin < 1.0)
5 {
6     dw1 += -y * x1;
7     dw2 += -y * x2;
8     db  += -y;
9 }
```

The complete source code (`main.c`) and dataset (`loan_dataset.csv`) are included in the project folder submitted alongside this report. This is a linear SVM-type classifier implemented using hinge-loss-based batch gradient updates (not a library-based SMO implementation).

2.10 Graphical User Interface

A Windows GUI was developed with input fields for Income and Credit Score, along with PREDICT and CLEAR buttons, and a result display area showing LOAN APPROVED or LOAN REJECTED.

2.11 Test Cases

Table 5: Test Cases

Income	Credit Score	Prediction
30	600	LOAN REJECTED
40	625	LOAN REJECTED
70	750	LOAN APPROVED
85	800	LOAN APPROVED
50	670	Borderline

2.12 Results and Analysis

Applicants with relatively higher income and credit scores were generally classified as Approved, while those with lower values were classified as Rejected, consistent with the learned decision boundary.

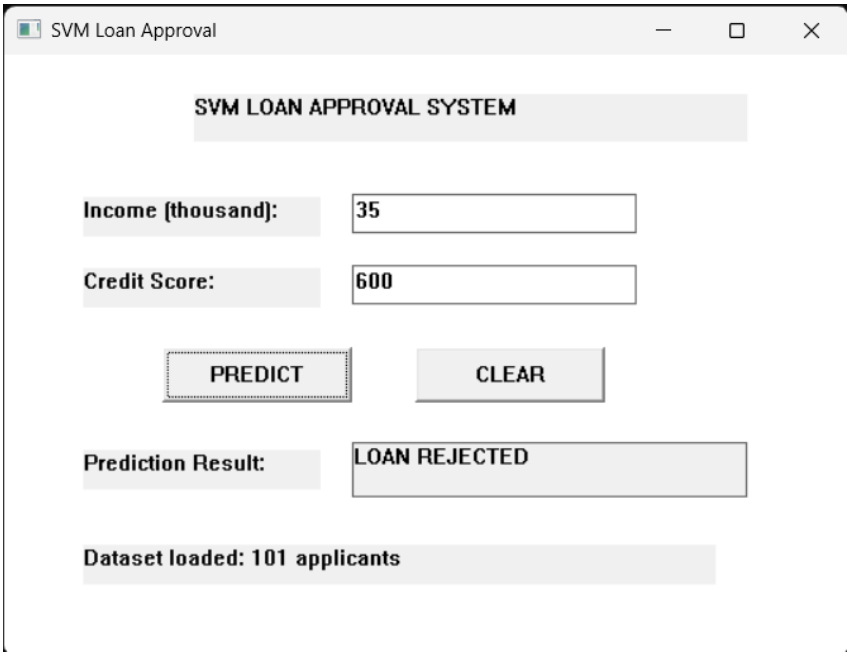


Figure 2: Decision boundary separating Approved and Rejected applicants

2.13 Conclusion

In this experiment, a Support Vector Machine (SVM) classifier was successfully implemented to predict loan approval based on income and credit score. The input features were normalized before training, and a hinge-loss-based learning process was used to obtain the classification boundary. A simple Windows GUI was developed to allow users to enter applicant information without using command-line input.

3 Problem 3: K-Means Student Performance Clustering

3.1 Objective

The objective of this problem is to implement the K-Means clustering algorithm to group students into three performance categories — **LOW**, **MEDIUM**, and **HIGH** performance — based on their study hours and exam scores.

3.2 Input and Output

- **Input:** Study Hours, Exam Score
- **Output:** LOW PERFORMANCE, MEDIUM PERFORMANCE, or HIGH PERFORMANCE

3.3 Algorithm Used

K-Means is an **unsupervised machine learning algorithm** used for clustering. Unlike KNN and SVM, the dataset used here has no predefined class labels. The algorithm groups students into $K = 3$ clusters purely based on the similarity between their Study Hours and Exam Score values. After clustering, the resulting groups are ranked and labeled as Low, Medium, and High performance according to their centroid values.

3.4 Dataset Description

The dataset file `student_cluster_dataset.csv` contains 100 student records with two numerical attributes: Study Hours and Exam Score. No performance label is present in the dataset. A sample is shown in Table 6.

Table 6: Sample of the Student Clustering Dataset (first 10 records)

Study Hours	Exam Score
1.0	35
1.5	40
2.0	42
2.5	48
3.0	50
6.0	68
7.0	73
9.0	84
11.0	92
13.0	98

Records: **Features:** Study Hours, Exam Score **Number of clusters (K):** 3

3.5 Workflow

START

↓

Load Student Dataset

↓

Initialize Centroids

↓

Calculate Distance to Each Centroid

↓

Assign Each Student to Nearest Cluster

↓

Update Centroids

↓

Repeat Until Iterations Complete

↓

Sort Clusters by Performance

↓

LOW / MEDIUM / HIGH

↓

END

3.6 Mathematical Formulation

3.6.1 Distance Calculation

The Euclidean distance between a student and a centroid is calculated as:

$$d = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2} \tag{6}$$

where x_1, x_2 are the Study Hours and Exam Score of the student, and c_1, c_2 are the corresponding centroid values.

3.6.2 Centroid Update

After assigning students to their nearest cluster, each centroid is updated as the mean of all points in that cluster:

$$c_j = \frac{1}{n_j} \sum_{i \in Cluster_j} x_i \tag{7}$$

where n_j is the number of students in cluster j.

3.6.3 Cluster Labeling

Since K-Means produces clusters without inherent meaning, the three final centroids are ranked by combined performance ($StudyHours + ExamScore$), and labeled as:

- Lowest centroid $\rightarrow$ LOW PERFORMANCE
- Middle centroid $\rightarrow$ MEDIUM PERFORMANCE
- Highest centroid $\rightarrow$ HIGH PERFORMANCE

3.7 Theory and Background

K-Means is an unsupervised learning algorithm used to partition a dataset into K distinct, non-overlapping clusters based on feature similarity. The algorithm iteratively assigns each data point to its nearest centroid and then recalculates the centroids as the mean of the points assigned to them. This process repeats until the centroids stabilize.
In this experiment, K-Means is applied to group students based on Study Hours and Exam Score, without using any predefined performance labels. Once clustering is complete, the resulting groups are interpreted and labeled as Low, Medium, and High performance based on their centroid values.

3.8 Implementation (C Code)

The algorithm was implemented in C using the Win32 API to build a graphical user interface. Key functions are shown below.

Listing 6: Euclidean Distance Calculation

```c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#define MAX_STUDENTS 100
#define K 3
#define ITERATIONS 100

#define ID_STUDY        201
#define ID_SCORE        202
#define ID_CLUSTER      203
#define ID_CLEAR        204
#define ID_RESULT       205
#define ID_STATUS       206
/* ============================================================
   DATA STRUCTURE
   ============================================================ */
typedef struct
{
    double studyHours;
    double examScore;
} Student;
/* ============================================================
   GLOBAL VARIABLES
   ============================================================ */
Student dataset[MAX_STUDENTS];
int studentCount = 0;

/* Centroids */
double centroidStudy[K];
double centroidScore[K];

/* Final cluster of each student */
int studentCluster[MAX_STUDENTS];
```

```
/* ============================================================
   LOAD DATASET
   ============================================================ */
int loadDataset()
{
    FILE *file;

    char line[200];

    int count = 0;

    file = fopen("student_cluster_dataset.csv", "r");

    if(file == NULL)
    {
        return 0;
    }

///main///
double calculateDistance(
    double studyHours,
    double examScore,
    double centroidStudyValue,
    double centroidScoreValue)
{
    double a = studyHours - centroidStudyValue;
    double b = examScore - centroidScoreValue;

    return sqrt((a * a) + (b * b));
}
```

Listing 7: Centroid Update Step

```
for(cluster = 0; cluster < K; cluster++)
{
    if(count[cluster] > 0)
    {
        centroidStudy[cluster] =
            sumStudy[cluster] / count[cluster];

        centroidScore[cluster] =
            sumScore[cluster] / count[cluster];
```

```
10          }
11  }
```

The complete source code (`main.c`) and dataset (`student_cluster_dataset.csv`) are included in the project folder submitted alongside this report. Clustering was performed with $K = 3$ over 100 iterations, followed by sorting the final centroids to assign Low/Medium/High labels.

3.9 Graphical User Interface

A Windows GUI was developed with input fields for Study Hours and Exam Score, along with CLUSTER and CLEAR buttons, and a result display area showing the predicted performance group.

3.10 Test Cases

Table 7: Test Cases

Study Hours	Exam Score	Predicted Group
2	45	LOW PERFORMANCE
6	70	MEDIUM PERFORMANCE
10	90	HIGH PERFORMANCE

3.11 Results and Analysis

The K-Means algorithm successfully grouped the 100 students into three clusters purely based on the similarity of their Study Hours and Exam Score, without using any predefined labels. Students with lower study hours and exam scores were grouped into the Low Performance cluster, while those with higher values formed the High Performance cluster, with an intermediate group forming the Medium Performance cluster.

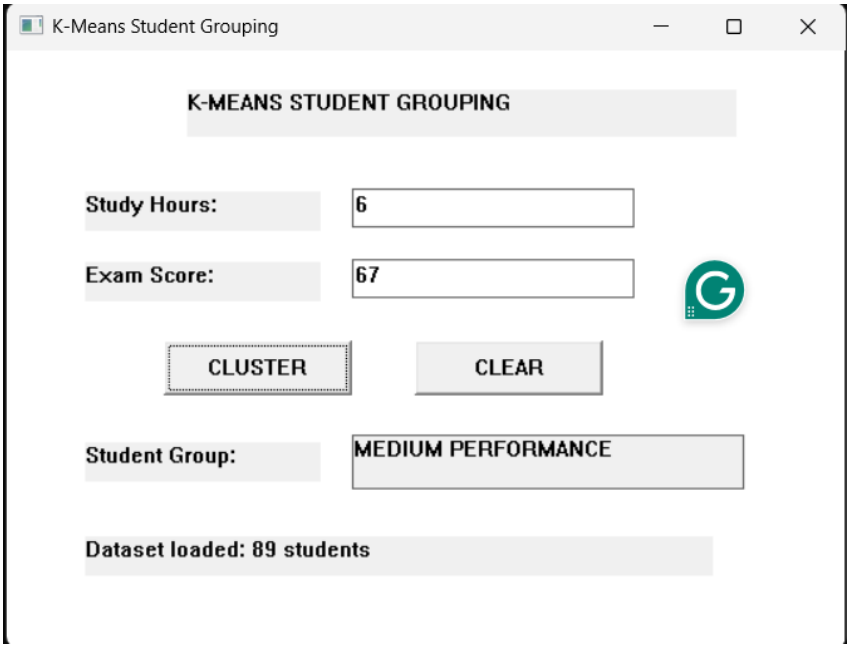


Figure 3: Scatter plot showing the three student performance clusters

3.12 Conclusion

In this experiment, the K-Means clustering algorithm was successfully implemented to group students into Low, Medium, and High performance categories based on their study hours and exam scores. Since K-Means is an unsupervised algorithm, no predefined labels were used during clustering; the final clusters were interpreted based on their centroid values. A Windows-based GUI was developed to allow users to input student data and view the predicted performance group directly.

4 Problem 4: Decision Tree Based Weather Activity Prediction

4.1 Objective

The objective of this problem is to implement a Decision Tree classifier to predict whether an outdoor activity should be played (**PLAY**) or not (**DON'T PLAY**) based on Temperature, Humidity, and Wind Speed.

4.2 Input and Output

- **Input:** Temperature (°C), Humidity (%), Wind Speed (km/h)
- **Output:** PLAY or DON'T PLAY

4.3 Algorithm Used

A Decision Tree is a supervised machine learning algorithm that recursively splits the dataset based on feature values to build a tree-like model of decisions. Each internal node represents a test on a feature, each branch represents the outcome of the test, and each leaf node represents a class label (PLAY or DON'T PLAY). The best feature and threshold at each split are chosen using **Gini impurity**.

4.4 Dataset Description

The dataset file `weather_dataset.csv` contains 100 records with three input features (Temperature, Humidity, Wind Speed) and one target class (Result). A sample is shown in Table 8.

Table 8: Sample of the Weather Dataset (first 10 records)

Temperature (°C)	Humidity (%)	Wind Speed (km/h)	Result
13.8	89	24.5	DON'T PLAY
13.3	83	14.6	DON'T PLAY
32.1	41	15.7	PLAY
20.9	73	23.3	PLAY
22.5	81	24.5	DON'T PLAY
29.4	59	12.8	PLAY
16.7	68	18.2	DON'T PLAY
35.8	45	9.4	DON'T PLAY
25.6	66	17.1	PLAY
11.9	92	27.3	DON'T PLAY

Records: **Features:** Temperature, Humidity, Wind Speed **Classes:** PLAY, DON'T PLAY

4.5 Workflow

START

↓

Load Weather Dataset

↓

Calculate Gini Impurity for Candidate Splits

↓

Select Best Feature and Threshold

↓

Split Data into Left / Right Branches

↓

Repeat Recursively (until pure node or max depth)

↓

Build Decision Tree

↓

Enter New Weather Data

↓

Traverse Tree to a Leaf Node

↓

PLAY / DON'T PLAY

↓

END

4.6 Mathematical Formulation

4.6.1 Gini Impurity

For a set of records, the Gini impurity is calculated as:

$$Gini = 1 - p_{play}^2 - p_{dontplay}^2 \tag{8}$$

where p_{play} and $p_{dontplay}$ are the proportions of PLAY and DON'T PLAY records in the set.

4.6.2 Weighted Gini for a Split

For a candidate split producing a left subset (L) and a right subset (R):

$$Gini_{split} = \frac{|L|}{|L| + |R|} \cdot Gini(L) + \frac{|R|}{|L| + |R|} \cdot Gini(R) \tag{9}$$

The feature and threshold that minimize $Gini_{split}$ are selected at each node.

4.7 Theory and Background

A Decision Tree is a supervised learning algorithm that models decisions as a tree of feature-based tests. Starting from the root node, the algorithm searches over all features and possible threshold values to find the split that produces the purest child subsets, measured using Gini impurity. This process is applied recursively to build the tree until the data in a node belongs to a single class or a maximum depth is reached. Once trained, a new sample is classified by traversing the tree from the root to a leaf node according to its feature values.
In this experiment, a Decision Tree is used to predict whether an outdoor activity should be played, based on Temperature, Humidity, and Wind Speed.

4.8 Implementation (C Code)

The algorithm was implemented in C using the Win32 API to build a graphical user interface. Key functions are shown below.

Listing 8: Gini Impurity Calculation

```
1 #include <windows.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <math.h>
6
```

```
#define MAX_STUDENTS 100
#define K 3
#define ITERATIONS 100

#define ID_STUDY        201
#define ID_SCORE        202
#define ID_CLUSTER      203
#define ID_CLEAR        204
#define ID_RESULT       205
#define ID_STATUS       206

/* ============================================================
   DATA STRUCTURE
   ============================================================ */

typedef struct
{
    double studyHours;
    double examScore;
} Student;


/* ============================================================
   GLOBAL VARIABLES
   ============================================================ */

Student dataset[MAX_STUDENTS];

int studentCount = 0;

/* Centroids */
double centroidStudy[K];
double centroidScore[K];

/* Final cluster of each student */
int studentCluster[MAX_STUDENTS];


/* ============================================================
   LOAD DATASET
   ============================================================ */
```

```
48
49  int loadDataset()
50  {
51      FILE *file;
52
53      char line[200];
54
55      int count = 0;
56
57      file = fopen("student_cluster_dataset.csv", "r");
58
59      if(file == NULL)
60      {
61          return 0;
62      }
63
64  ///main///
65  double giniImpurity(int *indices, int count)
66  {
67      if (count == 0) return 0.0;
68
69      int play = 0, dontPlay = 0;
70
71      for (int i = 0; i < count; i++)
72      {
73          if (dataset[indices[i]].result == 1)
74              play++;
75          else
76              dontPlay++;
77      }
78
79      double pPlay = (double)play / count;
80      double pDont = (double)dontPlay / count;
81
82      return 1.0 - (pPlay * pPlay) - (pDont * pDont);
83  }
```

Listing 9: Tree Traversal for Prediction

```
1  if (node->isLeaf)
2      return node->prediction;
3
```

```
if (value <= node->threshold)
    return predictTree(node->left, temperature, humidity, windSpeed)
   ;
else
    return predictTree(node->right, temperature, humidity, windSpeed
   );
```

The complete source code (`main.c`) and dataset (`weather_dataset.csv`) are included in the project folder submitted alongside this report. The tree was trained with a maximum depth of 3, using Gini impurity to select the best feature and threshold at each node.

4.9 Graphical User Interface

A Windows GUI was developed with input fields for Temperature, Humidity, and Wind Speed, along with PREDICT and CLEAR buttons, and a result display area showing PLAY or DON'T PLAY.

4.10 Test Cases

Table 9: Test Cases

Temperature	Humidity	Wind Speed	Prediction
25	60	10	PLAY
12	90	25	DON'T PLAY
30	55	12	PLAY
35	80	25	DON'T PLAY

4.11 Results and Analysis

The Decision Tree correctly classified new weather conditions by recursively testing feature thresholds learned from the training data. Moderate temperatures with lower humidity and wind speed generally resulted in PLAY, while high humidity or high wind speed conditions generally resulted in DON'T PLAY.

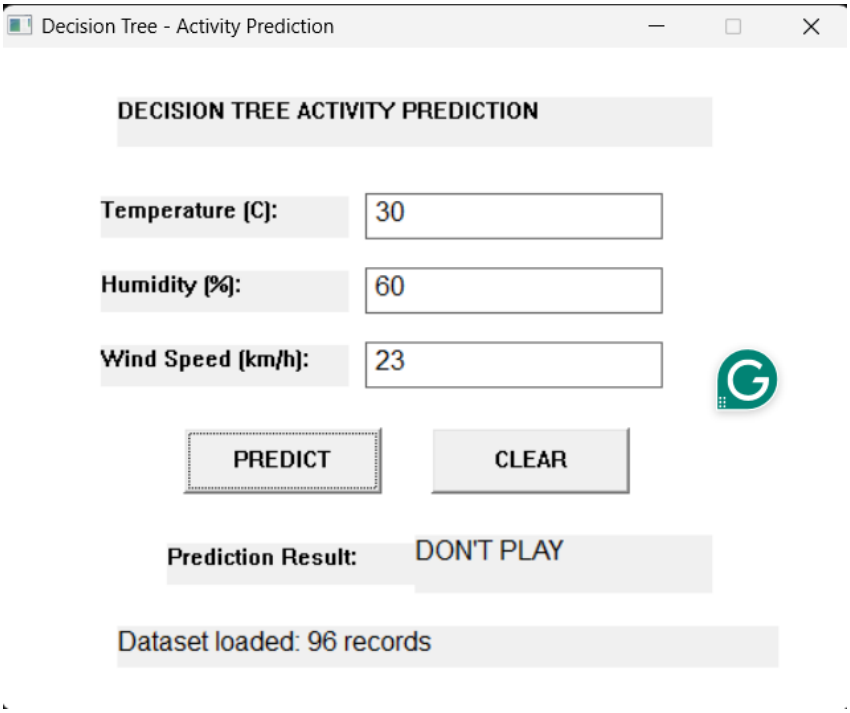


Figure 4: Structure of the trained decision tree showing root, decision, and leaf nodes

4.12 Conclusion

In this experiment, a Decision Tree classifier was successfully implemented to predict outdoor activity based on Temperature, Humidity, and Wind Speed. The tree was built using Gini impurity to select the best splits at each node, up to a maximum depth of three. A Windows-based GUI was developed to allow users to enter weather conditions and view the predicted PLAY/DON'T PLAY decision directly.

5 Problem 5: Perceptron Based Apple vs Orange Recognition

5.1 Objective

The objective of this problem is to implement a single-layer Perceptron (based on the classical Hagan fruit classification problem) to classify a fruit as **APPLE** or **ORANGE** based on its Shape, Texture, and Weight sensor readings.

5.2 Input and Output

- **Input:** Shape, Texture, Weight (each encoded as $+1$ or -1)
- **Output:** APPLE ($+1$) or ORANGE (-1)

Table 10: Sensor Encoding

Sensor	**+1**	**-1**
Shape	Round	Elliptical
Texture	Smooth	Rough
Weight	>1 pound	<1 pound

5.3 Algorithm Used

The Perceptron is one of the earliest supervised learning algorithms for binary classification. It computes a weighted sum of the inputs, passes it through a symmetric hard-limit activation function, and compares the output with the target class. If the output does not match the target, the weights and bias are updated; this process repeats until the training patterns are correctly classified.

This problem follows the classical formulation from Hagan's *Neural Network Design*, where the two training prototypes are:

$$p_{orange} = [\,1,\ -1,\ -1\,], \quad target = -1 \tag{10}$$

$$p_{apple} = [\,1,\ 1,\ -1\,], \quad target = +1 \tag{11}$$

5.4 Workflow

START

↓

Initialize Weights and Bias to Zero

↓

Calculate Weighted Sum (Net Input)

↓

Apply Activation Function (Hard Limit)

↓

Compare Output with Target

↓

Update Weights and Bias if Misclassified

↓

Repeat Until No Errors

↓

Enter New Fruit Sensor Values

↓

APPLE / ORANGE

↓

END

5.5 Mathematical Formulation

5.5.1 Net Input

$$n = w_1 x_1 + w_2 x_2 + w_3 x_3 + b \tag{12}$$

where x_1, x_2, x_3 are the Shape, Texture, and Weight values, and w_1, w_2, w_3, b are the weights and bias.

5.5.2 Activation Function

A symmetric hard-limit function is used:

$$a = \text{hardlims}(n) = \begin{cases} +1, & n \geq 0 \\ -1, & n < 0 \end{cases} \tag{13}$$

5.5.3 Weight Update Rule

$$w_{new} = w_{old} + e \cdot x, \qquad b_{new} = b_{old} + e \tag{14}$$

where $e = target - output$ is the classification error for a given training pattern.

5.6 Theory and Background

The Perceptron is a fundamental supervised learning algorithm used for binary linear classification. It consists of a single neuron that computes a weighted sum of its inputs and applies a hard-limit activation function to produce an output of $+1$ or -1. During training, the weights are adjusted whenever the predicted output does not match the desired target, using the perceptron learning rule. Training continues until all patterns are correctly classified or a maximum number of epochs is reached.
In this experiment, the Perceptron is trained on two prototype patterns representing an orange and an apple, based on their Shape, Texture, and Weight sensor readings, following the classical formulation presented in Hagan's *Neural Network Design*.

5.7 Training Process

The Perceptron was trained using the two labeled prototype patterns (Orange and Apple) shown above, starting from zero weights and bias, for up to 100 epochs or until both patterns were classified correctly.

Table 11: Training Patterns

Shape	Texture	Weight	Target
1	-1	-1	ORANGE (-1)
1	1	-1	APPLE $(+1)$

5.8 Implementation (C Code)

The algorithm was implemented in C using the Win32 API to build a graphical user interface. Key functions are shown below.

Listing 10: Symmetric Hard-Limit Activation Function

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
double w1 = 0.0;
double w2 = 0.0;
double w3 = 0.0;

/* Bias */
double bias = 0.0;
```

```
10
11  /* GUI controls */
12  HWND hShape;
13  HWND hTexture;
14  HWND hWeight;
15
16  HWND hResult;
17  HWND hDetails;
18
19  /* ================= HARD LIMIT FUNCTION ================= */
20
21  /*
22      Symmetric hard limit:
23
24      n >= 0   -> +1
25      n < 0    -> -1
26  */
27
28  int hardLimit(double n)
29  {
30      if (n >= 0)
31          return 1;
32
33      return -1;
34  }
35
36  /* ================= PERCEPTRON PREDICTION ================= */
37
38  int predictFruit(
39      int shape,
40      int texture,
41      int weight)
42  {
43      double net;
44
45      /*
46          n = Wp + b
47
48          n =
49          w1*shape +
50          w2*texture +
```

```
        w3*weight +
        bias
    */

    net =
        w1 * shape +
        w2 * texture +
        w3 * weight +
        bias;

    return hardLimit(net);

    ///main///
int hardLimit(double n)
{
    if (n >= 0)
        return 1;

    return -1;
}
```

Listing 11: Perceptron Weight Update Rule

```
error = targetOrange - prediction;

if (error != 0)
{
    w1 = w1 + error * orange[0];
    w2 = w2 + error * orange[1];
    w3 = w3 + error * orange[2];
    bias = bias + error;
}
```

The complete source code (`main.c`) is included in the project folder submitted alongside this report. No external dataset file is required, since the Perceptron is trained directly on the two prototype vectors defined by the problem.

5.9 Graphical User Interface

A Windows GUI was developed with dropdown selectors for Shape, Texture, and Weight, along with PREDICT and CLEAR buttons, and a result display area showing APPLE or ORANGE along with the computed net input value.

5.10 Test Cases

Table 12: Test Cases

Shape	Texture	Weight	Net Input	Prediction
Round $(+1)$	Rough (-1)	< 1 lb (-1)	—	ORANGE
Round $(+1)$	Smooth $(+1)$	< 1 lb (-1)	—	APPLE

5.11 Results and Analysis

After training, the Perceptron correctly classified both prototype patterns, producing ORANGE for the pattern $[1, -1, -1]$ and APPLE for the pattern $[1, 1, -1]$, consistent with the target labels used during training.

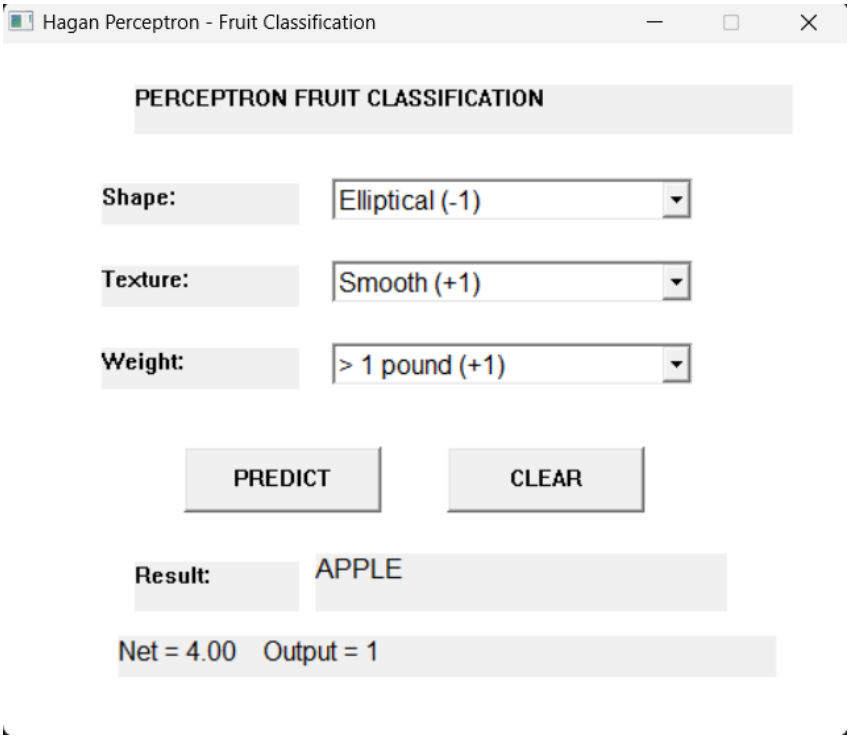


Figure 5: Decision boundary separating Apple and Orange classes

5.12 Conclusion

In this experiment, a single-layer Perceptron was successfully implemented to classify fruits as Apple or Orange based on Shape, Texture, and Weight sensor readings, following the classical Hagan fruit classification problem. The Perceptron learning rule was used to iteratively adjust the weights and bias until both training prototypes were correctly classified. A Windows-based GUI was developed to allow users to select sensor values and view the predicted classification directly.